Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Ethical Hacking & Defense - TP6 - Offline Password Cracking Attacks
4ème année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :	HATHOUTI Mohammed Taha JIDAL Ilyas
Filière :	Cybersécurité
Année :	2026/2027
Enseignant :	M. SEBBAR & M. ZEROUAL
Date :	6 novembre 2026

Table des matières

Objectifs **2**
Précautions Légales et Éthiques 2

1 Environnement de Travail **3**
1.1 Préparation de l'environnement 3

2 John the Ripper **3**
2.1 Présentation de John the Ripper 3
2.2 Test 1 : Attaque par dictionnaire 4
2.2.1 Principe 4
2.2.2 Exécution de l'attaque 4
2.3 Test 2 : Attaque hybride 5
2.3.1 Principe 5
2.3.2 Exécution de l'attaque 5
2.4 Test 3 : Attaque combinée 6
2.4.1 Principe 6
2.4.2 Exécution de l'attaque 6
2.4.3 Comparaison des trois méthodes 7

3 Test de l'attaque sur un système UNIX **8**
3.1 Création des utilisateurs 8
3.1.1 Principe 8
3.1.2 Création du compte user2 8
3.1.3 Création du compte user1 8
3.2 Attribution des mots de passe 9
3.2.1 Définition du mot de passe de user1 9
3.2.2 Configuration de la validité du mot de passe de user2 9
3.3 Analyse des fichiers /etc/passwd et /etc/shadow 10
3.3.1 Affichage du fichier /etc/shadow 10
3.3.2 Affichage du fichier /etc/passwd 10
3.4 Préparation des empreintes avec unshadow 11
3.4.1 Principe 11
3.4.2 Génération du fichier mypasswd 11
3.5 Audit des mots de passe avec John the Ripper 12
3.5.1 Exécution de John sur mypasswd 12
3.5.2 Interprétation des résultats 12
3.5.3 Comparaison des deux comptes 13

4 Synthèse et enseignements **13**
4.1 Comparaison des techniques d'attaque 13
4.2 Importance du stockage sécurisé des mots de passe 14
4.3 Bonnes pratiques de sécurité 14

Conclusion **14**

Objectifs

L'objectif de ce TP est d'étudier les mécanismes de stockage et de protection des mots de passe sous Linux, ainsi que les principales techniques permettant de les retrouver à partir de leurs empreintes cryptographiques. Pour cela, nous utilisons **John the Ripper**, un outil d'audit de mots de passe capable de réaliser différentes attaques hors ligne.

Ce travail permet notamment de comprendre la différence entre une attaque par dictionnaire, une attaque hybride et une attaque combinée, mais également d'étudier les fichiers système `/etc/passwd` et `/etc/shadow`, qui interviennent dans la gestion des comptes utilisateurs sous Linux.

Ce TP est organisé en deux grandes parties :

- **Partie 1** : John the Ripper ;
  - **Test 1** : Préparation de l'environnement et attaque par dictionnaire ;
  - **Test 2** : Attaque hybride avec les règles de substitution ;
  - **Test 3** : Attaque combinée et observation des résultats ;
- **Partie 2** : Test des attaques sur un système UNIX ;
  - **Test 1** : Création de comptes utilisateurs ;
  - **Test 2** : Attribution des mots de passe et gestion de leur validité ;
  - **Test 3** : Analyse des fichiers `/etc/passwd` et `/etc/shadow` ;
  - **Test 4** : Génération d'un fichier de mots de passe avec `unshadow` ;
  - **Test 5** : Audit des mots de passe avec John the Ripper.

Précautions Légales et Éthiques

IMPORTANT : Les techniques présentées dans ce TP peuvent être utilisées pour compromettre des comptes utilisateurs. Leur utilisation doit donc rester strictement limitée à des environnements de laboratoire et à des systèmes pour lesquels une autorisation explicite a été obtenue. L'ensemble des manipulations réalisées dans ce compte rendu concerne des comptes locaux créés spécifiquement pour ce TP sur notre propre machine.

1 Environnement de Travail

Pour réaliser ce TP, nous avons utilisé notre environnement Linux habituel, sous **Ubuntu 24.04**, avec l'outil **John the Ripper** dans sa version 1.8.0, conformément aux indications de l'énoncé.

L'ensemble des manipulations a été effectué dans le répertoire de travail suivant :

```
~/Bureau/UIR/4A/S7/Ethical_Hacking/TP6/
```

L'environnement utilisé est composé des éléments suivants :

- **Système d'exploitation :** Ubuntu 24.04 ;
- **Outil d'audit :** John the Ripper 1.8.0 ;
- **Dictionnaire :** `password.lst`, fourni avec John the Ripper ;
- **Fichier de test :** `crack_these_please.txt`, contenant les empreintes de 50 utilisateurs ;
- **Utilisateurs UNIX :** deux comptes locaux créés spécifiquement pour la seconde partie du TP, `user1` et `user2`.

1.1 Préparation de l'environnement

Après avoir téléchargé l'archive de John the Ripper, nous avons procédé à son extraction puis accédé au répertoire contenant les exécutables et les fichiers nécessaires :

```
mkdir LAB6
cd LAB6

wget www.openwall.com/john/j/john-1.8.0.tar.xz

tar -xvf john-1.8.0.tar.xz

cd john-1.8.0/run
```

Le fichier `crack_these_please.txt`, fourni dans le cadre du TP, a ensuite été placé dans le répertoire `run`. Il contient les empreintes de 50 utilisateurs différents, identifiés de `crack01` à `crack50`.

Ces empreintes sont associées à des mots de passe de niveaux de difficulté variables, certains étant particulièrement simples tandis que d'autres sont plus complexes.

2 John the Ripper

2.1 Présentation de John the Ripper

John the Ripper est un outil open source spécialisé dans l'audit de sécurité des mots de passe. Il permet notamment de tester la résistance des mots de passe en comparant les empreintes cryptographiques disponibles avec des candidats générés à partir de différentes méthodes.

Dans le cadre de ce TP, nous utilisons trois approches principales :

- **Attaque par dictionnaire :** les mots contenus dans un fichier de dictionnaire sont testés successivement ;

— **Attaque hybride :** des règles de transformation sont appliquées aux mots du dictionnaire afin de générer différentes variantes ;

— **Attaque combinée :** John utilise plusieurs méthodes de recherche, notamment les attaques par dictionnaire, hybrides et la recherche incrémentale.

L'objectif est d'observer les différences entre ces méthodes, notamment en matière de nombre de mots de passe retrouvés et de temps d'exécution.

2.2 Test 1 : Attaque par dictionnaire

2.2.1 Principe

Une attaque par dictionnaire consiste à tester une liste prédéfinie de mots de passe potentiels sur les empreintes disponibles.

Contrairement à une recherche exhaustive, cette méthode ne génère pas toutes les combinaisons possibles de caractères. Elle s'appuie sur un ensemble de mots susceptibles d'avoir été choisis par les utilisateurs.

Dans notre cas, le dictionnaire utilisé est le fichier `password.lst`, fourni avec John the Ripper. Il contient environ 3000 mots de passe potentiels.

Nous avons commencé par examiner son contenu :

```
more password.lst
```

2.2.2 Exécution de l'attaque

Nous avons ensuite lancé l'attaque par dictionnaire sur les 50 empreintes contenues dans le fichier fourni :

```
john --wordlist=password.lst crack_these_please.txt
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP6/john-1.8.0/run$ john --wordlist=password.lst crack_these_please.txt
Loaded 50 password hashes with 50 different salts (descrypt, traditional crypt(3) [DES 128/128 SSE2])
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
money            (crack21)
pass             (crack24)
test             (crack29)
blue             (crack03)
bonjour          (crack04)
cowboy           (crack07)
dog              (crack10)
www              (crack43)
www              (crack44)
hello            (crack14)
japan            (crack16)
11g 0:00:00:00 100% 275.0g/s 88550p/s 4427Kc/s 4427KC/s 123456..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

FIGURE 1 – Résultat de l'attaque par dictionnaire avec John the Ripper

Analyse des résultats :

L'exécution indique que John a correctement chargé les 50 empreintes, associées à 50 sels différents.

Au cours de cette attaque, plusieurs mots de passe ont été retrouvés. Parmi les résultats affichés, on retrouve notamment :

— `money` pour `crack21` ;

— `pass` pour `crack24` ;

— `test` pour `crack29` ;

— `blue` pour `crack03` ;

— `bonjour` pour `crack04` ;

— `cowboy` pour `crack07` ;

— `dog` pour `crack10` ;

— `hello` pour `crack14` ;

— `japan` pour `crack16`.

On remarque également plusieurs utilisateurs ayant choisi des variantes très simples telles que `www`.

L'attaque s'est terminée avec le statut `Session completed`, indiquant que le parcours du dictionnaire est arrivé à son terme.

Le débit affiché est d'environ 275 mots de passe testés par seconde, avec un total de 50 empreintes initialement chargées.

Observation : cette première expérience montre que les mots de passe courants, les mots simples et les termes fréquemment utilisés sont particulièrement vulnérables à une attaque par dictionnaire.

2.3 Test 2 : Attaque hybride

2.3.1 Principe

Une attaque hybride reprend le principe de l'attaque par dictionnaire, mais ajoute des règles de transformation permettant de générer plusieurs variantes à partir d'un même mot.

Ces transformations peuvent notamment inclure :

— Le remplacement de certaines lettres par des chiffres ou des caractères similaires ;

— L'ajout de chiffres ou de caractères à la fin d'un mot ;

— La modification de la casse ;

— La combinaison de plusieurs transformations sur un même candidat.

Cette approche permet de tester des mots de passe qui ne figurent pas directement dans le dictionnaire, mais qui restent construits à partir de mots courants.

2.3.2 Exécution de l'attaque

Pour activer les règles de transformation, nous avons utilisé l'option `-rules` :

```
john --wordlist=password.lst --rules crack_these_please.txt
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP6/john-1.8.0/run$ john --wordlist=password.lst --rules crack_these_please.txt
Loaded 50 password hashes with 50 different salts (descrypt, traditional crypt(3) [DES 128/128 SSE2])
Remaining 39 password hashes with 39 different salts
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
wwwwww           (crack47)
1g 0:00:00:00 100% 4.545g/s 633022p/s 24352Kc/s 24352KC/s bulled..Sssing
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

FIGURE 2 – Résultat de l'attaque hybride avec activation des règles

Analyse des résultats :

Avant de commencer cette nouvelle attaque, John indique que 39 empreintes restent à tester. Cela signifie que les résultats de l'attaque précédente sont conservés dans son fichier de session et que les empreintes déjà résolues ne nécessitent pas d'être testées à nouveau.

Cette fois, John retrouve notamment le mot de passe `wwwwww` associé à l'utilisateur `crack47`.

Le résultat est particulièrement intéressant puisqu'il s'agit d'une variante construite à partir d'un caractère très simple, répétée plusieurs fois.

L'exécution se termine rapidement avec le statut `Session completed`. Le débit affiché atteint environ 4,545 millions de candidats par seconde.

Observation : l'attaque hybride permet de retrouver des mots de passe qui ne correspondent pas nécessairement à une entrée exacte du dictionnaire. L'application de règles augmente donc considérablement le nombre de candidats testés et les possibilités de recherche.

2.4 Test 3 : Attaque combinée

2.4.1 Principe

John the Ripper dispose également d'un mode de recherche combiné permettant d'utiliser plusieurs méthodes successivement.

Cette approche ne se limite plus aux mots présents dans le dictionnaire ou à leurs variantes. Elle peut également recourir à une recherche incrémentale, qui explore différentes combinaisons de caractères.

L'objectif est donc de retrouver des mots de passe plus difficiles à identifier avec les deux méthodes précédentes.

2.4.2 Exécution de l'attaque

Nous avons lancé le mode combiné avec la commande suivante :

```
1 john crack_these_please.txt
```

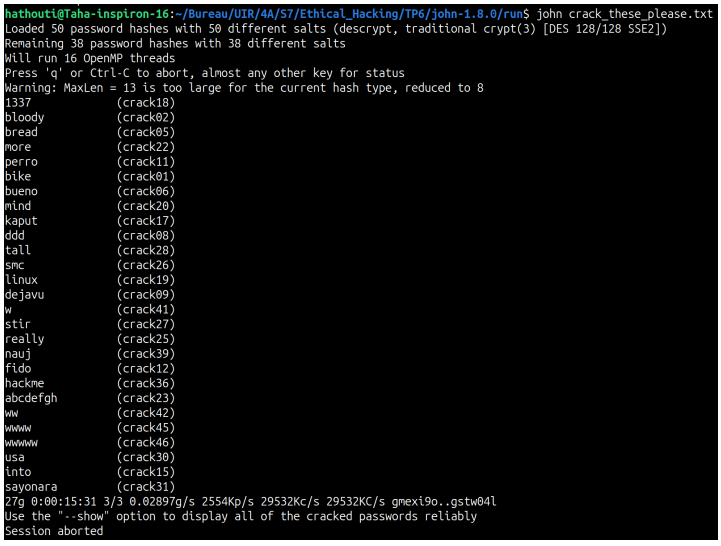


FIGURE 3 – Exécution de l'attaque combinée avec John the Ripper

Analyse des résultats :

Au lancement, John indique que 38 empreintes restent à traiter, après prise en compte des résultats obtenus lors des deux attaques précédentes.

Une observation importante apparaît dès le début de l'exécution :

```
Warning: MaxLen = 13 is too large for the current hash type,
reduced to 8
```

John signale que la longueur maximale initialement prévue est trop importante pour le type d'empreinte utilisé et qu'elle est automatiquement ramenée à 8 caractères.

Plusieurs mots de passe supplémentaires sont retrouvés au cours de cette exécution, notamment :

— `1337` pour `crack18` ;

— `bloody` pour `crack02` ;

— `bread` pour `crack05` ;

— `perro` pour `crack11` ;

— `bike` pour `crack01` ;

— `bueno` pour `crack06` ;

— `linux` pour `crack19` ;

— `hackme` pour `crack36` ;

— `abcdefgh` pour `crack23` ;

— `sayonara` pour `crack31`.

L'exécution atteint ensuite une durée de 15 minutes et 31 secondes. À ce moment, John affiche une progression de 27 groupes de candidats sur les 50 attendus, mais la session est interrompue manuellement.

Le statut affiché est donc `Session aborted` et non `Session completed`.

Observation : contrairement aux deux premières méthodes, l'attaque combinée peut nécessiter beaucoup plus de temps de calcul. Elle permet cependant d'explorer un espace de recherche plus large et de retrouver des mots de passe qui ne sont pas présents directement dans le dictionnaire.

2.4.3 Comparaison des trois méthodes

Les résultats obtenus permettent de distinguer clairement les trois approches utilisées :

Critère	**Dictionnaire**	**Hybride**	**Combinée**
Principe	Mots prédéfinis	Mots transformés	Plusieurs méthodes de recherche
Option utilisée	`-wordlist`	`-wordlist -rules`	Aucune option supplémentaire
Temps observé	Quelques secondes	Quelques secondes	15 min 31 s avant interruption
Résultat	Plusieurs mots simples retrouvés	Variantes retrouvées	Mots supplémentaires retrouvés
État final	Terminée	Terminée	Interrompue manuellement

TABLE 1 – Comparaison des attaques réalisées avec John the Ripper

Bilan de la première partie :

Les expériences montrent que la résistance d'un mot de passe dépend fortement de sa construction. Un mot de passe simple, présent dans un dictionnaire ou construit à partir d'un terme courant, peut être retrouvé très rapidement.

À l'inverse, les mots de passe nécessitant l'exploration d'un espace de recherche plus important demandent davantage de temps et de ressources.

3 Test de l'attaque sur un système UNIX

3.1 Création des utilisateurs

3.1.1 Principe

Dans cette seconde partie, nous allons appliquer les techniques étudiées précédemment à deux comptes utilisateurs créés localement sur notre système Linux.

L'objectif est de comprendre comment les informations relatives aux comptes et aux mots de passe sont stockées, puis d'utiliser John the Ripper pour réaliser un audit sur les empreintes correspondantes.

Les deux comptes concernés sont :

— `user1` ;

— `user2`.

3.1.2 Création du compte user2

Nous avons commencé par créer le premier compte utilisateur avec la commande :

```
sudo useradd user2
```



FIGURE 4 – Création du compte utilisateur user2

La commande `useradd` permet de créer un nouvel utilisateur dans le système. Les informations associées à ce compte sont notamment enregistrées dans le fichier `/etc/passwd`.

3.1.3 Création du compte user1

Nous avons ensuite créé le deuxième compte avec les options suivantes :

```
sudo useradd -m -s /bin/bash -c "User" user1
```



FIGURE 5 – Création du compte utilisateur user1

Les différentes options utilisées ont les rôles suivants :

— `-m` : crée automatiquement le répertoire personnel de l'utilisateur ;

— `-s /bin/bash` : définit Bash comme interpréteur de commandes par défaut ;

— `-c "User"` : ajoute une description au compte.

Le compte `user1` dispose donc d'un répertoire personnel situé dans `/home/user1` et utilise Bash comme shell.

3.2 Attribution des mots de passe

Après avoir créé les deux comptes, nous avons procédé à leur configuration.

3.2.1 Définition du mot de passe de user1

Pour attribuer un mot de passe au premier utilisateur, nous avons exécuté :

```
1 sudo passwd user1
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP6/john-1.8.0/run$ sudo passwd user1
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
```

FIGURE 6 – Définition du mot de passe de user1

Le système demande de saisir le nouveau mot de passe puis de le confirmer. Une fois l'opération terminée, le message `passwd : mot de passe mis à jour avec succès` confirme que le changement a été effectué.

3.2.2 Configuration de la validité du mot de passe de user2

Pour le deuxième utilisateur, nous avons exécuté :

```
1 sudo passwd -n 60 -x 90 -w 7 user2
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP6/john-1.8.0/run$ sudo passwd -n 60 -x 90 -w 7 user2
passwd : mot de passe changé.
```

FIGURE 7 – Configuration des paramètres de validité du mot de passe de user2

Contrairement à la commande précédente, cette commande ne définit pas directement un nouveau mot de passe. Elle configure les paramètres de durée de validité du mot de passe :

— `-n 60` : impose une durée minimale de 60 jours avant de pouvoir modifier le mot de passe ;

— `-x 90` : fixe une durée maximale de validité de 90 jours ;

— `-w 7` : demande au système d'avertir l'utilisateur 7 jours avant l'expiration du mot de passe.

Une attribution effective du mot de passe de `user2` a également été nécessaire par la suite, car la commande précédente ne faisait que modifier les paramètres de vieillissement du mot de passe.

3.3 Analyse des fichiers /etc/passwd et /etc/shadow

3.3.1 Affichage du fichier /etc/shadow

Le fichier `/etc/shadow` contient les informations relatives aux mots de passe des utilisateurs locaux. Contrairement à l'ancien fonctionnement où les empreintes pouvaient être présentes dans `/etc/passwd`, elles sont désormais stockées séparément dans ce fichier, dont l'accès est réservé aux utilisateurs privilégiés.

Nous avons affiché son contenu avec la commande :

```
sudo more /etc/shadow
```

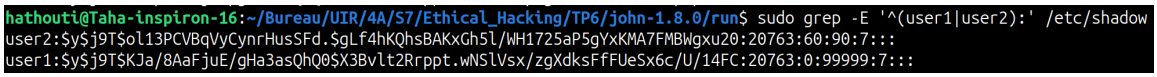


FIGURE 8 – Affichage des entrées de user1 et user2 dans /etc/shadow

Les deux entrées affichées suivent la structure générale suivante :

```
user:hash:dernier_changement:min:max:avertissement:inactivite:
   expiration:
```

Dans notre cas, l'information importante est l'entrée qui contient une empreinte commençant par `$y$`, correspondant à l'algorithme de hachage yescrypt.

3.3.2 Affichage du fichier /etc/passwd

Nous avons ensuite examiné le fichier `/etc/passwd`, qui contient les informations générales relatives aux comptes utilisateurs :

```
more /etc/passwd
```

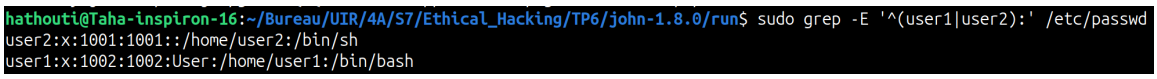


FIGURE 9 – Affichage des entrées de user1 et user2 dans /etc/passwd

Contrairement au fichier `/etc/shadow`, le fichier `/etc/passwd` ne contient pas directement les empreintes des mots de passe. La lettre `x` présente dans le deuxième champ indique que les informations correspondantes sont stockées dans `/etc/shadow`.

Chaque ligne suit la structure suivante :

```
user:password:UID:GID:commentaire:home:shell
```

Les principaux champs sont :

- **user :** nom du compte utilisateur ;
- **password :** généralement remplacé par `x`, indiquant que l'empreinte est stockée dans `/etc/shadow` ;
- **UID :** identifiant numérique de l'utilisateur ;
- **GID :** identifiant numérique du groupe principal ;

— **commentaire :** informations descriptives sur le compte ;

— **home :** chemin du répertoire personnel ;

— **shell :** interpréteur de commandes utilisé par l'utilisateur.

Dans notre cas, les informations relevées sont les suivantes :

Utilisateur	UID	GID	Répertoire personnel	Shell
user2	1001	1001	`/home/user2`	`/bin/sh`
user1	1002	1002	`/home/user1`	`/bin/bash`

TABLE 2 – Informations des deux comptes dans /etc/passwd

Conclusion intermédiaire : les deux fichiers ont des rôles complémentaires. Le fichier `/etc/passwd` contient les informations générales nécessaires à l'identification des comptes, tandis que `/etc/shadow` conserve les empreintes cryptographiques et les paramètres de validité des mots de passe.

3.4 Préparation des empreintes avec unshadow

3.4.1 Principe

Pour permettre à John the Ripper d'effectuer un audit des mots de passe Linux, nous devons combiner les informations présentes dans les fichiers `/etc/passwd` et `/etc/shadow`.

L'utilitaire **unshadow**, fourni avec John the Ripper, permet précisément de réaliser cette opération.

Il associe les informations générales des comptes aux empreintes cryptographiques correspondantes afin de générer un fichier dans un format exploitable par John.

3.4.2 Génération du fichier mypasswd

Nous avons utilisé la commande suivante :

```
sudo unshadow /etc/passwd /etc/shadow > mypasswd
```

Le fichier généré contient les informations des comptes du système ainsi que leurs empreintes.

Pour limiter l'audit aux deux utilisateurs créés dans le cadre de ce TP, nous avons conservé uniquement leurs entrées dans le fichier de travail :

```
sudo unshadow /etc/passwd /etc/shadow | \
grep -E '^(user1|user2):' > mypasswd
```

Cette opération permet de conserver les informations nécessaires pour `user1` et `user2`, sans inclure les autres comptes présents sur la machine.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP6/john-1.8.0/run$ cat mypasswd
user2:$y$j9T$ol13PCVBqVyCynrHusSFd.$gLf4hKQhsBAKxGh5l/WH1725aP5gYxKMA7FMBWgxu20:1001:1001::/home/user2:/bin/sh
user1:$y$j9T$KJa/8AaFjuE/gHa3asQhQ0$X3Bvlt2Rrppt.wNSlVsx/zgXdksFfFUeSx6c/U/14FC:1002:1002:User:/home/user1:/bin/bash
```

FIGURE 10 – Contenu du fichier mypasswd après génération des empreintes

Analyse du résultat :
Le fichier `mypasswd` contient désormais deux entrées :

— Une entrée correspondant à `user1`, contenant son empreinte yescrypt ;

— Une entrée correspondant à `user2`, contenant son empreinte après attribution effective de son mot de passe.

Les deux empreintes sont associées à des sels différents. Le sel est une valeur supplémentaire utilisée lors du calcul de l'empreinte afin d'éviter notamment que deux utilisateurs ayant choisi le même mot de passe possèdent nécessairement la même empreinte.

3.5 Audit des mots de passe avec John the Ripper

3.5.1 Exécution de John sur mypasswd

Une fois le fichier préparé, nous avons lancé John the Ripper sur les deux empreintes :

```
john --format=crypt mypasswd
```

L'option `-format=crypt` permet de préciser le format générique crypt utilisé pour l'interprétation des empreintes.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP6/john-1.8.0/run$ john --format=crypt mypasswd
Loaded 2 password hashes with 2 different salts (crypt, generic crypt(3) [?/64])
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
123456           (user2)
1g 0:00:02:34 34% 2/3 0.006452g/s 404.3p/s 405.4c/s 405.4C/s country0..ghost0
Use the "--show" option to display all of the cracked passwords reliably
Session aborted
```

FIGURE 11 – Exécution de John the Ripper sur les empreintes des deux utilisateurs

Analyse des résultats :

Au démarrage, John confirme avoir chargé deux empreintes associées à deux sels différents :

```
Loaded 2 password hashes with 2 different salts
(crypt, generic crypt(3))
```

Le premier résultat obtenu concerne `user2`. John retrouve son mot de passe et l'affiche dans la sortie :

```
123456 (user2)
```

En revanche, aucun résultat n'est affiché pour `user1` pendant la durée de cette expérience.

Après environ 2 minutes et 34 secondes, la progression indiquée est de 34 %, avec deux empreintes restantes dans la session. Nous avons alors interrompu manuellement l'exécution afin de passer à la suite du travail.

Le statut final affiché est `Session aborted`.

3.5.2 Interprétation des résultats

Les résultats obtenus mettent en évidence une différence importante entre les deux comptes :

— **user2 :** le mot de passe a été retrouvé rapidement. Il s'agit d'un mot de passe très simple, facilement identifiable par une recherche parmi les candidats testés ;

— **user1 :** aucun mot de passe n'a été retrouvé durant la période d'exécution observée. L'absence de résultat ne signifie cependant pas que le mot de passe est impossible à retrouver, mais uniquement que la recherche n'a pas abouti avant l'interruption.

Le mot de passe de `user1` est construit à partir d'un terme comportant une combinaison de lettres majuscules et minuscules ainsi que des chiffres. Cette construction rend sa recherche plus difficile avec une approche générique.

Il faut également tenir compte de l'algorithme utilisé pour le stockage des empreintes. L'entrée de `user1` dans `/etc/shadow` commence par `$y$`, ce qui indique l'utilisation de yescrypt.

Le coût de calcul associé à l'algorithme de hachage influence directement le nombre de candidats pouvant être testés pendant une période donnée.

3.5.3 Comparaison des deux comptes

Critère	**user1**	**user2**
Empreinte présente	Oui	Oui
Algorithme	Yescrypt	Yescrypt
Mot de passe retrouvé	Non durant le test	Oui
Temps d'exécution observé	Environ 2 min 34 s avant interruption	
Résultat	Non retrouvé	Retrouvé

TABLE 3 – Comparaison des résultats de l'audit des deux comptes

Observation : cette expérience illustre l'importance du choix d'un mot de passe suffisamment robuste. Un mot de passe simple peut être retrouvé rapidement, même lorsque son empreinte est protégée par un mécanisme de hachage. À l'inverse, un mot de passe moins prévisible augmente considérablement le coût de la recherche.

4 Synthèse et enseignements

Les différentes expériences réalisées permettent de dégager plusieurs observations concernant la sécurité des mots de passe.

4.1 Comparaison des techniques d'attaque

Les trois méthodes étudiées dans la première partie présentent des caractéristiques différentes :

— L'attaque par dictionnaire est particulièrement efficace contre les mots de passe courants et les expressions fréquemment utilisées ;

— L'attaque hybride améliore les résultats en générant des variantes à partir des mots existants ;

— L'attaque combinée permet d'explorer un espace de recherche plus important, mais peut nécessiter beaucoup plus de temps.

Les résultats obtenus sur les 50 empreintes montrent que plusieurs mots de passe simples ont pu être retrouvés rapidement, tandis que certaines recherches ont nécessité une durée plus importante.

4.2 Importance du stockage sécurisé des mots de passe

La seconde partie a permis de comprendre l'organisation des fichiers de gestion des utilisateurs sous Linux.

Le fichier `/etc/passwd` contient les informations générales relatives aux comptes, tandis que `/etc/shadow` conserve les empreintes cryptographiques et les paramètres de validité des mots de passe.

L'utilisation d'un algorithme adapté, comme yescrypt, permet d'augmenter le coût des tentatives de recherche hors ligne. Cependant, même un algorithme robuste ne peut pas compenser totalement le choix d'un mot de passe extrêmement faible.

4.3 Bonnes pratiques de sécurité

À partir des résultats obtenus, plusieurs recommandations peuvent être formulées :

— Éviter les mots de passe courants, les termes du dictionnaire et les suites de chiffres simples ;

— Privilégier des mots de passe suffisamment longs et difficiles à deviner ;

— Éviter de réutiliser le même mot de passe sur plusieurs services ;

— Utiliser des algorithmes de hachage adaptés au stockage des mots de passe, comme yescrypt ;

— Restreindre l'accès au fichier `/etc/shadow` aux utilisateurs autorisés ;

Conclusion

Ce TP nous a permis d'étudier concrètement les mécanismes d'audit des mots de passe à l'aide de **John the Ripper**, ainsi que les différentes méthodes utilisées pour retrouver des mots de passe à partir de leurs empreintes cryptographiques.

La première partie a permis de comparer trois approches : l'attaque par dictionnaire, l'attaque hybride et l'attaque combinée. Les résultats montrent que les mots de passe simples et prévisibles sont particulièrement vulnérables, tandis que les recherches plus complexes peuvent nécessiter davantage de temps et de ressources.

La seconde partie a permis d'appliquer ces notions à deux comptes utilisateurs créés sur notre système Ubuntu. L'analyse des fichiers `/etc/passwd` et `/etc/shadow` a mis en évidence la séparation entre les informations générales des comptes et les empreintes des mots de passe.

L'utilisation de l'utilitaire `unshadow` nous a ensuite permis de générer un fichier exploitable par John the Ripper afin de réaliser un audit sur les deux comptes.

Enfin, les résultats obtenus ont montré une différence significative entre les deux utilisateurs : le mot de passe de `user2` a été retrouvé rapidement, tandis que celui de `user1` n'a pas été retrouvé durant la période de test.

Cette expérience confirme qu'une politique de sécurité efficace repose à la fois sur le choix de mots de passe robustes, l'utilisation d'algorithmes de hachage adaptés et la protection des fichiers contenant les informations d'authentification.